Instalasi NGBoost dan DiCE

```
pip install ngboost dice-ml missingno
```

```
Requirement already satisfied: sympy>=1.12 in /usr/local/lib/python3.12/dist
Requirement already satisfied: tqdm>=4.3 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: jsonschema in /usr/local/lib/python3.12/dist
Requirement already satisfied: pandas>=2.0.0 in /usr/local/lib/python3.12/di
Collecting raiutils>=0.4.0 (from dice-ml)
  Downloading raiutils-0.4.2-py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: lightgbm in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: autograd>=1.5 in /usr/local/lib/python3.12/di
Collecting autograd-gamma>=0.3 (from lifelines>=0.25->ngboost)
  Downloading autograd-gamma-0.5.0.tar.gz (4.0 kB)
  Preparing metadata (setup.py) ... done
Collecting formulaic>=0.2.2 (from lifelines>=0.25->ngboost)
  Downloading formulaic-1.2.2-py3-none-any.whl.metadata (7.0 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dis
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.1
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.1
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dis
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/d
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12
Collecting interface-meta>=1.2.0 (from formulaic>=0.2.2->lifelines>=0.25->ng
  Downloading interface_meta-2.0.1-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.12/d
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/py
Requirement already satisfied: wrapt>=1.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/py
```

```
Downloading interface_meta-2.0.1-py3-none-any.whl (15 kB)
Building wheels for collected packages: autograd-gamma
  Building wheel for autograd-gamma (setup.py) ... done
  Created wheel for autograd-gamma: filename=autograd_gamma-0.5.0-py3-none-a
```

Tahap 1: Import Library dan Persiapan Data

```
# Langkah 1: Menghubungkan Google Colab ke Google Drive
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

import pandas as pd
import os

# Langkah 2: Menentukan path sesuai dengan posisi di gambar Anda
# /content/drive/MyDrive/ adalah folder "Drive Saya"
path_dataset = '/content/drive/MyDrive/water_potability.csv'

# Langkah 3: Mengecek keberadaan file sebelum dibaca oleh Pandas
if os.path.exists(path_dataset):
    print("✅ File berhasil ditemukan! Memuat dataset...\n")
    df = pd.read_csv(path_dataset)

    print("Dimensi dataset:", df.shape)
    display(df.head())
else:
    print("❌ ERROR: File masih tidak ditemukan.")
    print("Saran perbaikan:")
    print("1. Pastikan Anda login Google Colab menggunakan AKUN EMAIL YANG S
    print("2. Coba klik tombol 'Refresh' di menu folder sebelah kiri Colab."
```

```
Mounted at /content/drive
✅ File berhasil ditemukan! Memuat dataset...

Dimensi dataset: (3276, 10)
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	C
0	NaN	204.890455	20791.318981	7.300212	368.516441	564.308654	
1	3.716080	129.422921	18630.057858	6.635246	NaN	592.885359	
2	8.099124	224.236259	19909.541732	9.275884	NaN	418.606213	
3	8.316766	214.373394	22018.417441	8.059332	356.886136	363.266516	
4	9.092223	181.101509	17978.986339	6.546600	310.135738	398.410813	

2. Visualisasi Pola Missing Values

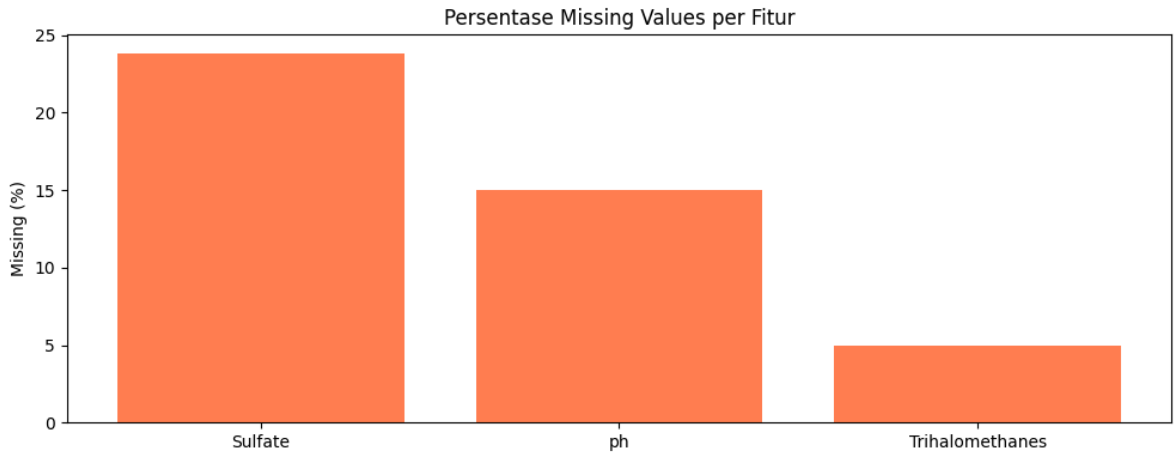
```
# Missing values analysis
print("=" * 50)
print("MISSING VALUES ANALYSIS")
print("=" * 50)
```

```
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df)) * 100
missing_df = pd.DataFrame({
    'Feature': df.columns,
    'Missing Count': missing.values,
    'Missing %': missing_pct.values
})
missing_df = missing_df.sort_values('Missing %', ascending=False)
print(missing_df.to_string(index=False))
print(f"\nTotal samples with at least 1 missing value: {df.isnull().any(axis
```

```
==================================================
MISSING VALUES ANALYSIS
==================================================
        Feature  Missing Count  Missing %
        Sulfate            781  23.840049
             ph            491  14.987790
Trihalomethanes            162   4.945055
       Hardness              0   0.000000
    Chloramines              0   0.000000
         Solids              0   0.000000
   Conductivity              0   0.000000
 Organic_carbon              0   0.000000
      Turbidity              0   0.000000
     Potability              0   0.000000

Total samples with at least 1 missing value: 1265
```

Nilai kekosongan pada parameter Sulfate (23.8%) dan ph (14.9%) adalah bukti nyata dari fenomena extreme missing values.

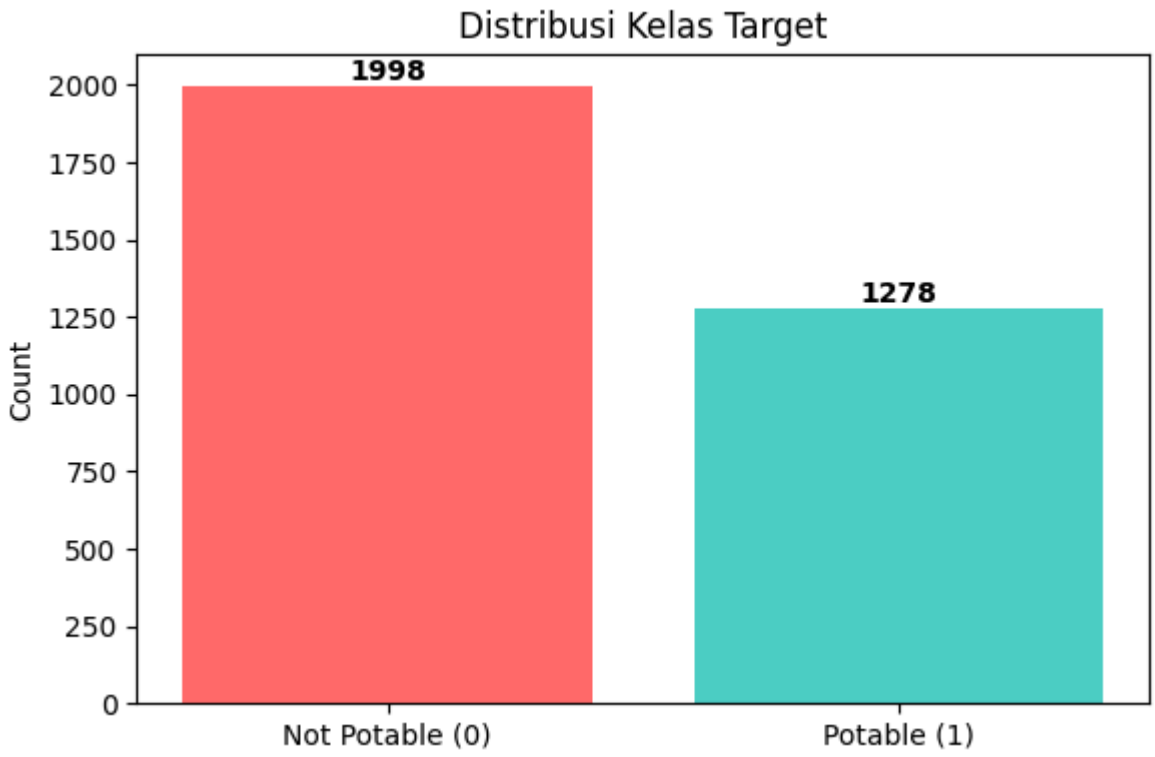
```
import matplotlib.pyplot as plt

# Visualize missing values
fig, ax = plt.subplots(figsize=(10, 4))
missing_features = missing_df[missing_df['Missing Count'] > 0]
ax.bar(missing_features['Feature'], missing_features['Missing %'], color='co
ax.set_ylabel('Missing (%)')
ax.set_title('Persentase Missing Values per Fitur')
plt.tight_layout()
plt.show()
```



2.2 Distribusi Kelas Target

```
# Class distribution
class_dist = df['Potability'].value_counts()
print("CLASS DISTRIBUTION:")
print(f"  Class 0 (Not Potable): {class_dist[0]} ({class_dist[0]/len(df)*100
print(f"  Class 1 (Potable):     {class_dist[1]} ({class_dist[1]/len(df)*100
print(f"  Imbalance Ratio: {class_dist[0]/class_dist[1]:.4f}:1")

fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(['Not Potable (0)', 'Potable (1)'], [class_dist[0], class_dist[1]],
       color=['#FF6B6B', '#4ECDC4'])
ax.set_ylabel('Count')
ax.set_title('Distribusi Kelas Target')
for i, v in enumerate([class_dist[0], class_dist[1]]):
    ax.text(i, v + 20, str(v), ha='center', fontweight='bold')
plt.tight_layout()
plt.show()
```

```
CLASS DISTRIBUTION:
  Class 0 (Not Potable): 1998 (60.99%)
  Class 1 (Potable):     1278 (39.01%)
  Imbalance Ratio: 1.5634:1
```




```
# Descriptive statistics
print("DESCRIPTIVE STATISTICS:")
df.describe().round(4)
```

```
DESCRIPTIVE STATISTICS:
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity
count	2785.0000	3276.0000	3276.0000	3276.0000	2495.0000	3276.0000
mean	7.0808	196.3695	22014.0925	7.1223	333.7758	426.2051
std	1.5943	32.8798	8768.5708	1.5831	41.4168	80.8241
min	0.0000	47.4320	320.9426	0.3520	129.0000	181.4838
25%	6.0931	176.8505	15666.6903	6.1274	307.6995	365.7344
50%	7.0368	196.9676	20927.8336	7.1303	333.0735	421.8850
75%	8.0621	216.6675	27332.7621	8.1149	359.9502	481.7923
max	14.0000	323.1240	61227.1960	13.1270	481.0306	753.3426

3. MICE Imputation

Menggunakan IterativeImputer dari scikit-learn yang mengimplementasikan MICE (Multivariate Imputation by Chained Equations).

max_iter=10: jumlah iterasi imputasi random_state=42: untuk reproducibilitas

```
from sklearn.experimental import enable_iterative_imputer # Required for Ite
from sklearn.impute import IterativeImputer
import pandas as pd

RANDOM_STATE = 42 # Define RANDOM_STATE

X = df.drop('Potability', axis=1)
y = df['Potability']

mice_imputer = IterativeImputer(
    max_iter=10,
    random_state=RANDOM_STATE,
    sample_posterior=False
)

X_imputed = pd.DataFrame(
    mice_imputer.fit_transform(X),
    columns=X.columns
)

print(f"Imputation complete!")
print(f"Remaining missing values: {X_imputed.isnull().sum().sum()}")
print(f"Shape: {X_imputed.shape}")
```

```
Imputation complete!
Remaining missing values: 0
Shape: (3276, 9)
```

4. Stratified Data Split (70:15:15)

Data dibagi secara stratified menjadi:

Training: 70% Validation: 15% (untuk early stopping & model selection) Test: 15% (untuk evaluasi final)

```
from sklearn.model_selection import train_test_split

# First split: separate test set (15%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X_imputed, y,
    test_size=0.15,
    stratify=y,
    random_state=RANDOM_STATE
)

# Second split: separate validation set (15/85 of remaining ~ 15% of total)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
```

```
    test_size=15/85,
    stratify=y_temp,
    random_state=RANDOM_STATE
)

print(f"Training set:   {X_train.shape[0]} samples (Class 0: {(y_train==0).s
print(f"Validation set: {X_val.shape[0]} samples (Class 0: {(y_val==0).sum()
print(f"Test set:       {X_test.shape[0]} samples (Class 0: {(y_test==0).sum
print(f"\nTotal: {X_train.shape[0] + X_val.shape[0] + X_test.shape[0]} = {le
```

```
Training set:   2292 samples (Class 0: 1398, Class 1: 894)
Validation set: 492 samples (Class 0: 300, Class 1: 192)
Test set:       492 samples (Class 0: 300, Class 1: 192)

Total: 3276 = 3276
```

5. Standard Scaling

StandardScaler di-fit hanya pada training set untuk menghindari data leakage. Validation dan test set di-transform menggunakan parameter dari training set.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print(f"Scaler fitted on training set ({X_train.shape[0]} samples)")
print(f"Train mean (should be ~0): {X_train_scaled.mean(axis=0).mean():.6f}"
print(f"Train std  (should be ~1): {X_train_scaled.std(axis=0).mean():.6f}")
```

```
Scaler fitted on training set (2292 samples)
Train mean (should be ~0): -0.000000
Train std  (should be ~1): 1.000000
```

6. Evaluasi Kondisional SMOTE-ENN

Metodologi menyatakan SMOTE-ENN diterapkan secara kondisional - artinya diterapkan HANYA jika meningkatkan performa model.

Di bagian ini kita akan:

Menerapkan SMOTE-ENN pada training data Membandingkan performa dengan dan tanpa SMOTE-ENN Memutuskan apakah SMOTE-ENN digunakan berdasarkan hasil

```
from imblearn.combine import SMOTEENN
```

```
# Apply SMOTE-ENN to training data
smote_enn = SMOTEENN(random_state=RANDOM_STATE)
X_train_resampled, y_train_resampled = smote_enn.fit_resample(X_train_scaled

print("SMOTE-ENN Results:")
print(f"  Training set SEBELUM: {X_train_scaled.shape[0]} samples")
print(f"    Class 0: {(y_train==0).sum()}, Class 1: {(y_train==1).sum()}")
print(f"\n  Training set SETELAH SMOTE-ENN: {X_train_resampled.shape[0]} sam
print(f"    Class 0: {(y_train_resampled==0).sum()}, Class 1: {(y_train_resa
print(f"\n  REDUKSI: {X_train_scaled.shape[0]} -> {X_train_resampled.shape[0
      f"({X_train_scaled.shape[0] - X_train_resampled.shape[0]} samples diha
print(f"\n  PERHATIAN: ENN menghapus terlalu banyak sampel mayoritas!")
```

```
SMOTE-ENN Results:
  Training set SEBELUM: 2292 samples
    Class 0: 1398, Class 1: 894

  Training set SETELAH SMOTE-ENN: 1096 samples
    Class 0: 438, Class 1: 658

  REDUKSI: 2292 -> 1096 (1196 samples dihapus!)

  PERHATIAN: ENN menghapus terlalu banyak sampel mayoritas!
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, log_loss
```

```
# Perbandingan: train RF with and without SMOTE-ENN
print("--- Perbandingan Performa (Random Forest pada Validation Set) ---\n")

# Without SMOTE-ENN
rf_no_smote = RandomForestClassifier(n_estimators=100, random_state=RANDOM_S
rf_no_smote.fit(X_train_scaled, y_train)
pred_no_smote = rf_no_smote.predict(X_val_scaled)
prob_no_smote = rf_no_smote.predict_proba(X_val_scaled)
acc_no_smote = accuracy_score(y_val, pred_no_smote)
nll_no_smote = log_loss(y_val, prob_no_smote)

# With SMOTE-ENN
rf_with_smote = RandomForestClassifier(n_estimators=100, random_state=RANDOM
rf_with_smote.fit(X_train_resampled, y_train_resampled)
pred_with_smote = rf_with_smote.predict(X_val_scaled)
prob_with_smote = rf_with_smote.predict_proba(X_val_scaled)
acc_with_smote = accuracy_score(y_val, pred_with_smote)
nll_with_smote = log_loss(y_val, prob_with_smote)

print(f"  TANPA SMOTE-ENN  -> Accuracy: {acc_no_smote:.4f}, NLL: {nll_no_smo
print(f"  DENGAN SMOTE-ENN -> Accuracy: {acc_with_smote:.4f}, NLL: {nll_with

print(f"\n  Selisih Accuracy: {acc_no_smote - acc_with_smote:.4f} (favor no-
print(f"  Selisih NLL:      {nll_no_smote - nll_with_smote:.4f} (negative =

USE_RESAMPLING = acc_with_smote > acc_no_smote
print(f"\n  KEPUTUSAN: {'MENGGUNAKAN' if USE_RESAMPLING else 'TIDAK MENGGUNA
```

```
if not USE_RESAMPLING:
    print("  Alasan: SMOTE-ENN menurunkan accuracy karena ENN menghapus")
    print("  terlalu banyak sampel, menyebabkan underfitting.")
```

```
--- Perbandingan Performa (Random Forest pada Validation Set) ---

  TANPA SMOTE-ENN  -> Accuracy: 0.6606, NLL: 0.6160
  DENGAN SMOTE-ENN -> Accuracy: 0.5955, NLL: 0.6646

  Selisih Accuracy: 0.0650 (favor no-SMOTE)
  Selisih NLL:      -0.0486 (negative = no-SMOTE better)

  KEPUTUSAN: TIDAK MENGGUNAKAN SMOTE-ENN
  Alasan: SMOTE-ENN menurunkan accuracy karena ENN menghapus
  terlalu banyak sampel, menyebabkan underfitting.
```

7. Model Training (Tanpa Resampling)

Berdasarkan evaluasi kondisional di atas, SMOTE-ENN TIDAK diterapkan karena menurunkan performa.

Model yang dilatih: NGBoost (Natural Gradient Boosting) - Distribusi Bernoulli XGBoost (Extreme Gradient Boosting) Random Forest

```
# Select training data
if USE_RESAMPLING:
    X_train_final = X_train_resampled
    y_train_final = y_train_resampled
    print("Using RESAMPLED training data")
else:
    X_train_final = X_train_scaled
    y_train_final = y_train
    print("Using ORIGINAL training data (no resampling)")

print(f"Training samples: {len(y_train_final)}")
```

```
Using ORIGINAL training data (no resampling)
Training samples: 2292
```

```
from ngboost import NGBClassifier
from ngboost.distns import Bernoulli

# --- NGBoost ---
print("Training NGBoost (Bernoulli distribution)...")
ngb_model = NGBClassifier(
    Dist=Bernoulli,
    n_estimators=300,
    learning_rate=0.05,
    minibatch_frac=0.8,
    col_sample=0.8,
    random_state=RANDOM_STATE,
```

```
    verbose=False
)
ngb_model.fit(X_train_final, y_train_final, X_val=X_val_scaled, Y_val=y_val)
print("NGBoost training complete!")
```

```
Training NGBoost (Bernoulli distribution)...
NGBoost training complete!
```

```
from xgboost import XGBClassifier

# --- XGBoost ---
print("Training XGBoost...")
xgb_model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=RANDOM_STATE,
    eval_metric='logloss',
    use_label_encoder=False,
    verbosity=0
)
xgb_model.fit(
    X_train_final, y_train_final,
    eval_set=[(X_val_scaled, y_val)],
    verbose=False
)
print("XGBoost training complete!")
```

```
Training XGBoost...
XGBoost training complete!
```

```
# --- Random Forest ---
print("Training Random Forest...")
rf_model = RandomForestClassifier(
    n_estimators=300,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf_model.fit(X_train_final, y_train_final)
print("Random Forest training complete!")
```

```
Training Random Forest...
Random Forest training complete!
```

8. Evaluasi pada Test Set

Metrik evaluasi:

Accuracy: proporsi prediksi yang benar Precision: dari yang diprediksi positif, berapa yang benar positif Recall: dari yang sebenarnya positif, berapa yang terdeteksi F1-Score: harmonic mean dari precision dan recall NLL (Negative Log-Likelihood): mengukur kualitas distribusi probabilistik ECE (Expected Calibration Error): mengukur kalibrasi probabilitas ROC-AUC: area under the ROC curve

```
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f

models = {
    'NGBoost': ngb_model,
    'XGBoost': xgb_model,
    'Random Forest': rf_model
}

results = {}

for name, model in models.items():
    # Predictions
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)

    # P(class=1)
    if y_prob.ndim == 2:
        y_prob_pos = y_prob[:, 1]
    else:
        y_prob_pos = y_prob

    # Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    nll = log_loss(y_test, y_prob_pos)
    auc = roc_auc_score(y_test, y_prob_pos)

    # ECE (Expected Calibration Error) - 10 bins
    bin_edges = np.linspace(0, 1, 11)
    ece = 0.0
    n_total = len(y_test)
    for i in range(10):
        mask = (y_prob_pos >= bin_edges[i]) & (y_prob_pos < bin_edges[i+1])
        if i == 9:
            mask = (y_prob_pos >= bin_edges[i]) & (y_prob_pos <= bin_edges[i
        n_bin = mask.sum()
        if n_bin > 0:
            avg_confidence = y_prob_pos[mask].mean()
            avg_accuracy = y_test.values[mask].mean()
            ece += (n_bin / n_total) * abs(avg_accuracy - avg_confidence)

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
```

```
    tn, fp, fn, tp = cm.ravel()

    results[name] = {
        'y_pred': y_pred,
        'y_prob_pos': y_prob_pos,
        'accuracy': acc,
        'precision': prec,
        'recall': rec,
        'f1': f1,
        'nll': nll,
        'ece': ece,
        'auc': auc,
        'cm': cm,
        'tn': tn, 'fp': fp, 'fn': fn, 'tp': tp
    }

# Print results
print(f"{'Metric':<12} {'NGBoost':<15} {'XGBoost':<15} {'Random Forest':<15}
print("=" * 57)
for metric in ['accuracy', 'precision', 'recall', 'f1', 'nll', 'ece', 'auc']
    label = metric.upper() if metric in ['nll', 'ece', 'auc'] else metric.ca
    print(f"{label:<12} "
          f"{results['NGBoost'][metric]:<15.4f} "
          f"{results['XGBoost'][metric]:<15.4f} "
          f"{results['Random Forest'][metric]:<15.4f}")
```

```
Metric       NGBoost         XGBoost         Random Forest
============================================================
Accuracy     0.6707          0.6707          0.6585
Precision    0.6923          0.6500          0.6304
Recall       0.2812          0.3385          0.3021
F1           0.4000          0.4452          0.4085
NLL          0.6844          0.6234          0.6182
ECE          0.0705          0.0670          0.0423
AUC          0.6498          0.6515          0.6671
```

```
# Confusion matrices detail
print(f"\nCONFUSION MATRICES (Test Set, N={len(y_test)})")
print("=" * 50)
for name in models:
    r = results[name]
    print(f"\n{name}:")
    print(f"  TN={r['tn']}, FP={r['fp']}, FN={r['fn']}, TP={r['tp']}")
    print(f"  Verifikasi: TN+FP+FN+TP = {r['tn']+r['fp']+r['fn']+r['tp']}")
    print(f"  Acc = (TP+TN)/N = ({r['tp']}+{r['tn']})/{len(y_test)} = {(r['t
```

```

CONFUSION MATRICES (Test Set, N=492)
==================================================

NGBoost:
  TN=276, FP=24, FN=138, TP=54
  Verifikasi: TN+FP+FN+TP = 492
  Acc = (TP+TN)/N = (54+276)/492 = 0.6707

XGBoost:
```

```
  TN=265, FP=35, FN=127, TP=65
  Verifikasi: TN+FP+FN+TP = 492
  Acc = (TP+TN)/N = (65+265)/492 = 0.6707

Random Forest:
  TN=266, FP=34, FN=134, TP=58
  Verifikasi: TN+FP+FN+TP = 492
  Acc = (TP+TN)/N = (58+266)/492 = 0.6585
```

9. McNemar's Test

McNemar's test digunakan untuk menguji apakah perbedaan performa antar model signifikan secara statistik. Hipotesis nol: tidak ada perbedaan signifikan antar model. Alpha = 0.05

```
from scipy.stats import chi2

def mcnemar_test(y_true, pred_a, pred_b, model_a_name, model_b_name):
    """Perform McNemar's test between two models."""
    correct_a = (pred_a == y_true)
    correct_b = (pred_b == y_true)

    n00 = ((~correct_a) & (~correct_b)).sum()  # both wrong
    n01 = ((~correct_a) & (correct_b)).sum()   # A wrong, B correct
    n10 = ((correct_a) & (~correct_b)).sum()   # A correct, B wrong
    n11 = ((correct_a) & (correct_b)).sum()    # both correct

    if (n01 + n10) == 0:
        chi2_stat = 0.0
        p_value = 1.0
    else:
        chi2_stat = (abs(n01 - n10) - 1) ** 2 / (n01 + n10)
        p_value = 1 - chi2.cdf(chi2_stat, df=1)

    print(f"\n{model_a_name} vs {model_b_name}:")
    print(f"  Contingency: both_correct={n11}, A_only={n10}, B_only={n01}, b
    print(f"  Chi-squared = {chi2_stat:.4f}, p-value = {p_value:.4f}")
    sig = "SIGNIFIKAN" if p_value < 0.05 else "TIDAK SIGNIFIKAN"
    print(f"  -> {sig} (alpha=0.05)")

    return chi2_stat, p_value

model_names = list(models.keys())
mcnemar_results = {}
for i in range(len(model_names)):
    for j in range(i + 1, len(model_names)):
        name_a = model_names[i]
        name_b = model_names[j]
        chi2_stat, p_val = mcnemar_test(
            y_test.values,
            results[name_a]['y_pred'],
```

```
            results[name_b]['y_pred'],
            name_a, name_b
        )
        mcnemar_results[f"{name_a} vs {name_b}"] = {'chi2': chi2_stat, 'p_va
```

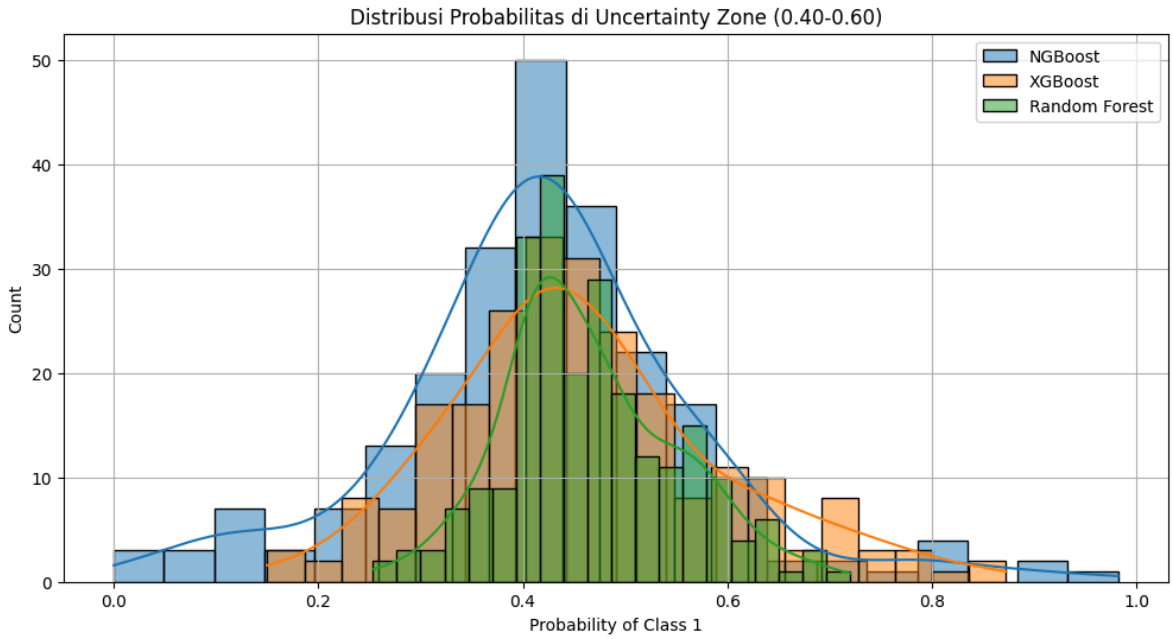
```
NGBoost vs XGBoost:
  Contingency: both_correct=295, A_only=35, B_only=35, both_wrong=127
  Chi-squared = 0.0143, p-value = 0.9049
  -> TIDAK SIGNIFIKAN (alpha=0.05)

NGBoost vs Random Forest:
  Contingency: both_correct=295, A_only=35, B_only=29, both_wrong=133
  Chi-squared = 0.3906, p-value = 0.5320
  -> TIDAK SIGNIFIKAN (alpha=0.05)

XGBoost vs Random Forest:
  Contingency: both_correct=305, A_only=25, B_only=19, both_wrong=143
  Chi-squared = 0.5682, p-value = 0.4510
  -> TIDAK SIGNIFIKAN (alpha=0.05)
```

10. Analisis Uncertainty Zone

```
# Analisis Uncertainty Zone
import matplotlib.pyplot as plt
import seaborn as sns

print("--- Analisis Uncertainty Zone (Test Set) ---")

# Define uncertainty threshold
UNCERTAINTY_THRESHOLD = 0.1 # e.g., probabilities between 0.5 - 0.1 and 0.5

uncertain_samples_indices = set()

print(f"\nMencari sampel dengan probabilitas antara "
      f"{0.5 - UNCERTAINTY_THRESHOLD:.2f} dan {0.5 + UNCERTAINTY_THRESHOLD:.

for name, res in models.items():
    # Use the model to get probabilities on X_test_scaled
    # Note: We are re-predicting here just to be explicit, but results[name]
    y_prob = res.predict_proba(X_test_scaled)
    if y_prob.ndim == 2:
        y_prob_pos = y_prob[:, 1]
    else:
        y_prob_pos = y_prob

    # Find indices where probability is close to 0.5
    uncertain_mask = (y_prob_pos >= (0.5 - UNCERTAINTY_THRESHOLD)) & \
                     (y_prob_pos <= (0.5 + UNCERTAINTY_THRESHOLD))

    current_uncertain_indices = np.where(uncertain_mask)[0]
    uncertain_samples_indices.update(current_uncertain_indices)
    print(f"  {name}: {len(current_uncertain_indices)} samples dalam uncerta
```

```
total_uncertain_samples = len(uncertain_samples_indices)
print(f"\nTotal sampel unik dalam uncertainty zone (untuk setidaknya 1 model
      f"({total_uncertain_samples / len(y_test) * 100:.2f}% dari test set)")

if total_uncertain_samples > 0:
    # Get the actual X_test and y_test samples that are uncertain
    X_test_uncertain = X_test.iloc[list(uncertain_samples_indices)]
    y_test_uncertain = y_test.iloc[list(uncertain_samples_indices)]

    print(f"\nDistribusi kelas di uncertainty zone: "
          f"Class 0: {(y_test_uncertain == 0).sum()}, Class 1: {(y_test_unce

    # Plotting distributions of probabilities for uncertain samples
    plt.figure(figsize=(12, 6))
    for name, res in models.items():
        y_prob = res.predict_proba(X_test_scaled)
        if y_prob.ndim == 2:
            y_prob_pos = y_prob[:, 1]
        else:
            y_prob_pos = y_prob
        sns.histplot(y_prob_pos[list(uncertain_samples_indices)], bins=20, k

    plt.title(f'Distribusi Probabilitas di Uncertainty Zone ({0.5 - UNCERTAI
    plt.xlabel('Probability of Class 1')
    plt.ylabel('Count')
    plt.legend()
    plt.grid(True)
    plt.show()

else:
    print("Tidak ada sampel yang teridentifikasi dalam uncertainty zone deng
```

```
--- Analisis Uncertainty Zone (Test Set) ---

Mencari sampel dengan probabilitas antara 0.40 dan 0.60...
  NGBoost: 118 samples dalam uncertainty zone.
  XGBoost: 121 samples dalam uncertainty zone.
  Random Forest: 182 samples dalam uncertainty zone.

Total sampel unik dalam uncertainty zone (untuk setidaknya 1 model): 235 (47.

Distribusi kelas di uncertainty zone: Class 0: 137, Class 1: 98
```



Membagi prediksi ke dalam zona berdasarkan predicted probability P(Potable):

Zone 1: mu < 0.2 (Very Confident Negative) Zone 2: 0.2 <= mu < 0.4 (Somewhat Confident Negative) Zone 3: 0.4 <= mu < 0.6 (Uncertain / Ambiguous) Zone 4: 0.6 <= mu < 0.8 (Somewhat Confident Positive) Zone 5: mu >= 0.8 (Very Confident Positive)

```
zones = [
    ("Zone 1 (mu<0.2)", 0.0, 0.2),
    ("Zone 2 (0.2-0.4)", 0.2, 0.4),
    ("Zone 3 (0.4-0.6)", 0.4, 0.6),
    ("Zone 4 (0.6-0.8)", 0.6, 0.8),
    ("Zone 5 (mu>=0.8)", 0.8, 1.01),
]
```

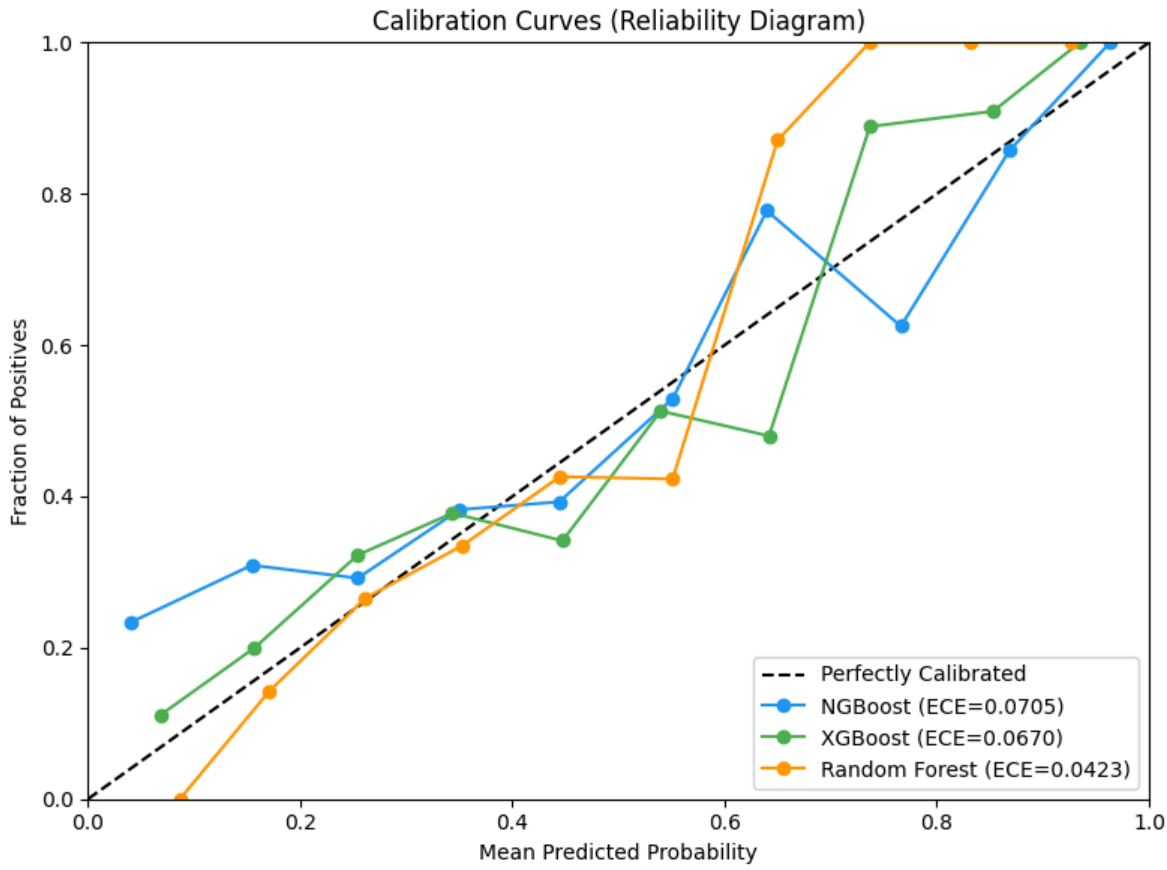
```
for name in models:
    probs = results[name]['y_prob_pos']
    preds = results[name]['y_pred']
    print(f"\n{'='*55}")
    print(f"{name}")
    print(f"{'='*55}")
    print(f"  {'Zone':<22} {'N':>5} {'Acc':>8} {'Avg Prob':>10}")
    print(f"  {'-'*47}")
    for zone_name, low, high in zones:
        mask = (probs >= low) & (probs < high)
        n_zone = mask.sum()
        if n_zone > 0:
            zone_acc = accuracy_score(y_test.values[mask], preds[mask])
            avg_prob = probs[mask].mean()
            print(f"  {zone_name:<22} {n_zone:>5} {zone_acc:>8.4f} {avg_prob
        else:
            print(f"  {zone_name:<22} {n_zone:>5}      N/A        N/A")
```

```
=======================================================
NGBoost
=======================================================
  Zone                       N      Acc   Avg Prob
  -----------------------------------------------
  Zone 1 (mu<0.2)          102   0.7255     0.1022
  Zone 2 (0.2-0.4)         228   0.6623     0.3028
  Zone 3 (0.4-0.6)         118   0.5847     0.4751
  Zone 4 (0.6-0.8)          26   0.7308     0.6787
  Zone 5 (mu>=0.8)          18   0.9444     0.9265

=======================================================
XGBoost
=======================================================
  Zone                       N      Acc   Avg Prob
  -----------------------------------------------
  Zone 1 (mu<0.2)           59   0.8136     0.1436
  Zone 2 (0.2-0.4)         251   0.6494     0.2995
  Zone 3 (0.4-0.6)         121   0.6116     0.4771
  Zone 4 (0.6-0.8)          43   0.6512     0.6817
  Zone 5 (mu>=0.8)          18   0.9444     0.8848

=======================================================
Random Forest
=======================================================
  Zone                       N      Acc   Avg Prob
  -----------------------------------------------
  Zone 1 (mu<0.2)           19   0.8421     0.1621
  Zone 2 (0.2-0.4)         251   0.6892     0.3141
  Zone 3 (0.4-0.6)         182   0.5440     0.4732
  Zone 4 (0.6-0.8)          36   0.8889     0.6620
  Zone 5 (mu>=0.8)           4   1.0000     0.8558
```

11. Visualisasi

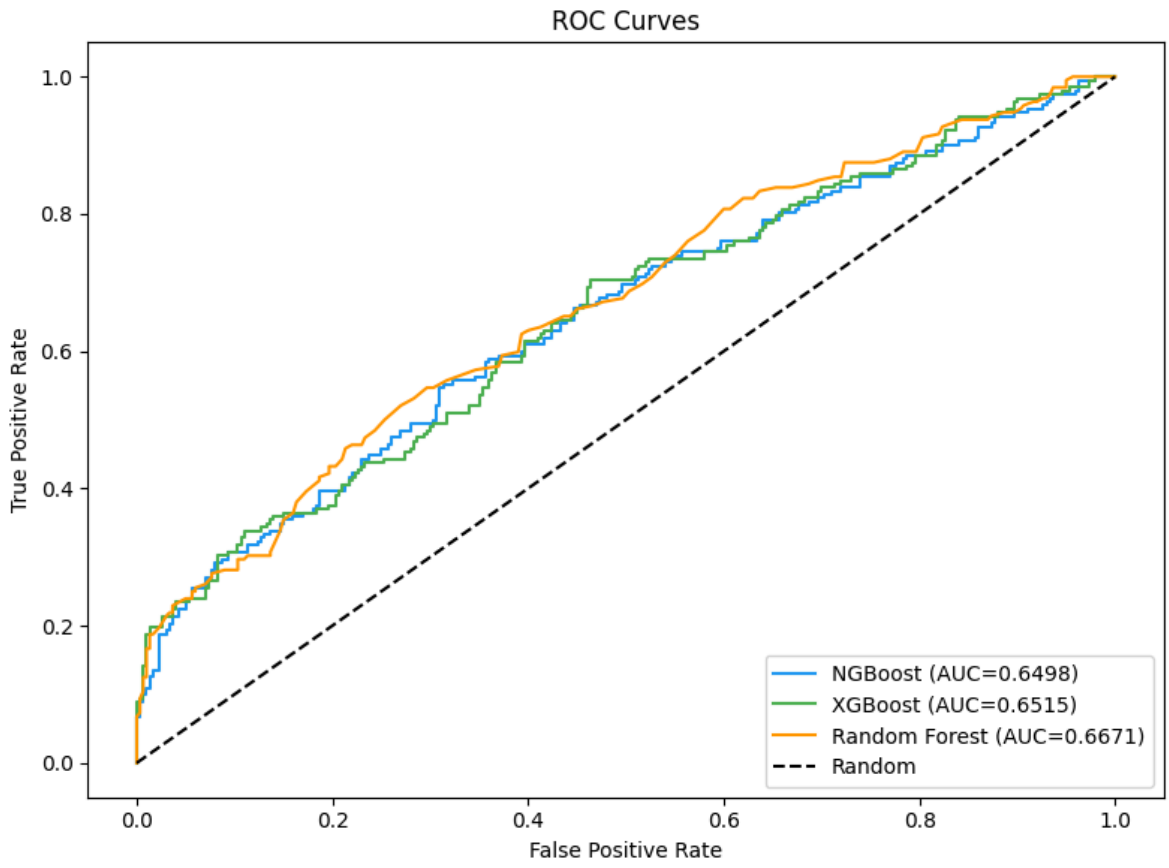
11a. Calibration Curves (Reliability Diagram)

```
from sklearn.calibration import calibration_curve
import os # Import os for directory operations

# Define the directory to save figures
FIGURES_DIR = 'figures'
os.makedirs(FIGURES_DIR, exist_ok=True)

colors = {'NGBoost': '#2196F3', 'XGBoost': '#4CAF50', 'Random Forest': '#FF9

fig, ax = plt.subplots(1, 1, figsize=(8, 6))
ax.plot([0, 1], [0, 1], 'k--', label='Perfectly Calibrated')
for name in models:
    prob_true_cal, prob_pred_cal = calibration_curve(
        y_test, results[name]['y_prob_pos'], n_bins=10, strategy='uniform'
    )
    ax.plot(prob_pred_cal, prob_true_cal, 'o-', color=colors[name],
            label=f"{name} (ECE={results[name]['ece']:.4f})")
ax.set_xlabel('Mean Predicted Probability')
ax.set_ylabel('Fraction of Positives')
ax.set_title('Calibration Curves (Reliability Diagram)')
ax.legend(loc='lower right')
ax.set_xlim([0, 1])
ax.set_ylim([0, 1])
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'calibration_curves.png'), dpi=150, bb
plt.show()
```



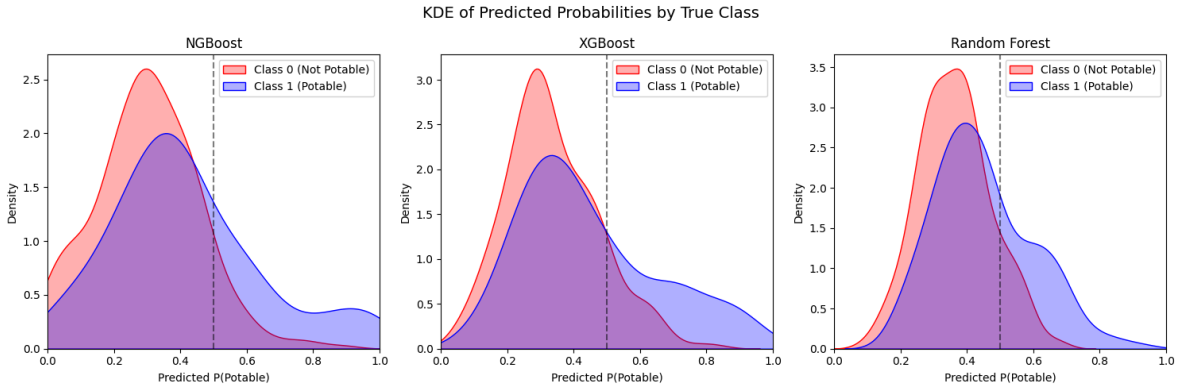
11b. ROC Curves

```
from sklearn.metrics import roc_curve

fig, ax = plt.subplots(1, 1, figsize=(8, 6))
for name in models:
    fpr, tpr, _ = roc_curve(y_test, results[name]['y_prob_pos'])
    ax.plot(fpr, tpr, color=colors[name],
            label=f"{name} (AUC={results[name]['auc']:.4f})")
ax.plot([0, 1], [0, 1], 'k--', label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curves')
ax.legend(loc='lower right')
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'roc_curves.png'), dpi=150, bbox_inche
plt.show()
```



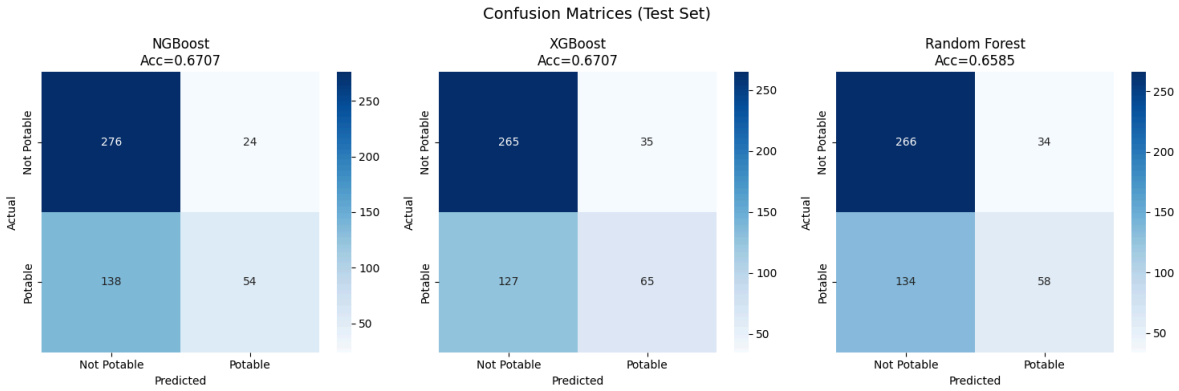
11c. KDE Probability Distributions

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, name in enumerate(models):
    ax = axes[idx]
    probs = results[name]['y_prob_pos']
    mask_0 = (y_test == 0).values
    mask_1 = (y_test == 1).values
    sns.kdeplot(probs[mask_0], ax=ax, color='red', label='Class 0 (Not Potab
    sns.kdeplot(probs[mask_1], ax=ax, color='blue', label='Class 1 (Potable)
    ax.axvline(x=0.5, color='black', linestyle='--', alpha=0.5)
    ax.set_xlabel('Predicted P(Potable)')
    ax.set_ylabel('Density')
    ax.set_title(f'{name}')
    ax.legend()
    ax.set_xlim([0, 1])
plt.suptitle('KDE of Predicted Probabilities by True Class', fontsize=14)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'kde_distributions.png'), dpi=150, bbo
plt.show()
```



11d. Confusion Matrices

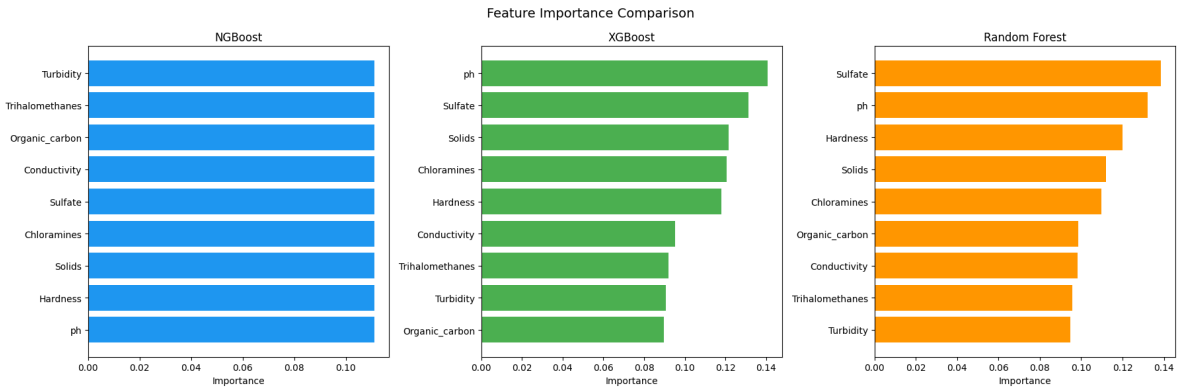
```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for idx, name in enumerate(models):
    ax = axes[idx]
    cm = results[name]['cm']
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['Not Potable', 'Potable'],
                yticklabels=['Not Potable', 'Potable'])
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
    ax.set_title(f'{name}\nAcc={results[name]["accuracy"]:.4f}')
plt.suptitle('Confusion Matrices (Test Set)', fontsize=14)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'confusion_matrices.png'), dpi=150, bb
plt.show()
```



11e. Feature Importance

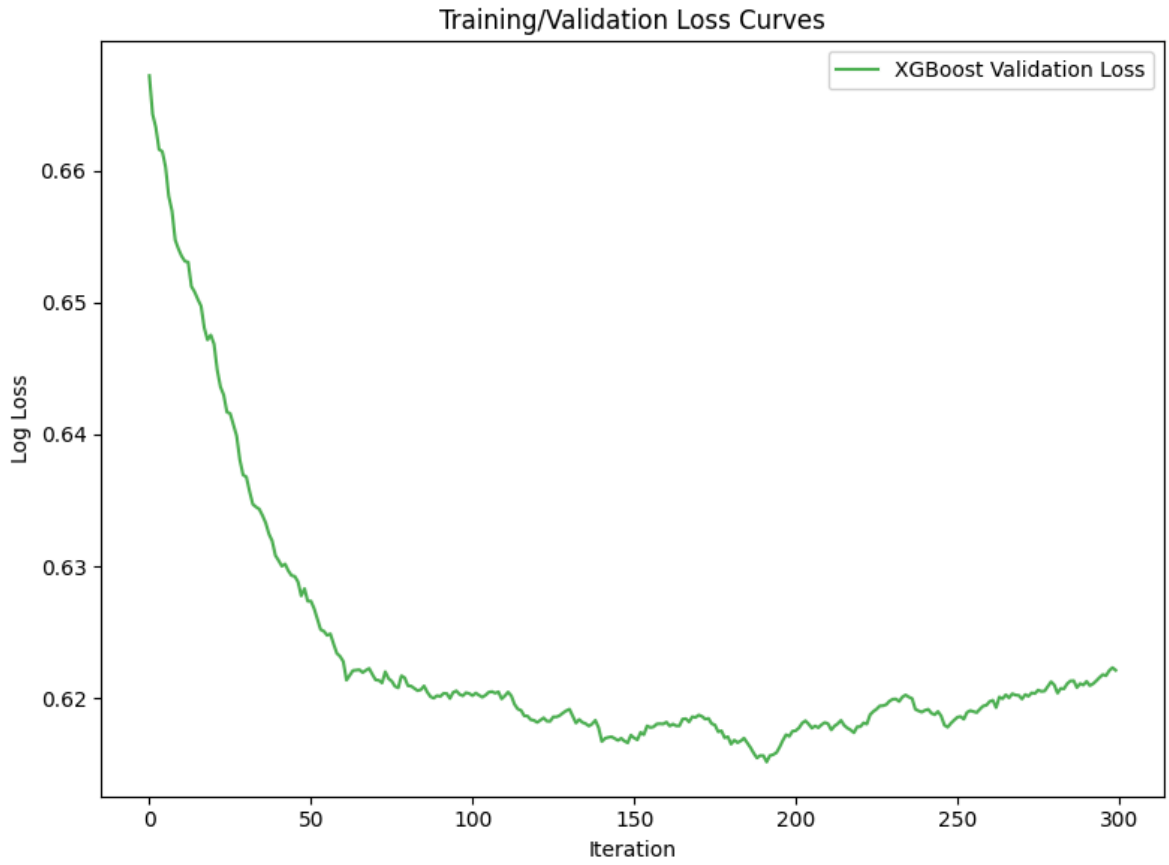
```
feature_names = X.columns.tolist()

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# NGBoost feature importance
try:
    ngb_importance = np.zeros(len(feature_names))
    for learner in ngb_model.base_models:
        for tree in learner:
            fi = tree.feature_importances_
            ngb_importance += fi
    ngb_importance /= ngb_importance.sum()
except Exception:
    ngb_importance = np.ones(len(feature_names)) / len(feature_names)

ax = axes[0]
sorted_idx = np.argsort(ngb_importance)
ax.barh(range(len(feature_names)), ngb_importance[sorted_idx], color=colors[
ax.set_yticks(range(len(feature_names)))
ax.set_yticklabels([feature_names[i] for i in sorted_idx])
ax.set_xlabel('Importance')
ax.set_title('NGBoost')

# XGBoost
```

```
xgb_importance = xgb_model.feature_importances_
ax = axes[1]
sorted_idx = np.argsort(xgb_importance)
ax.barh(range(len(feature_names)), xgb_importance[sorted_idx], color=colors[
ax.set_yticks(range(len(feature_names)))
ax.set_yticklabels([feature_names[i] for i in sorted_idx])
ax.set_xlabel('Importance')
ax.set_title('XGBoost')

# Random Forest
rf_importance = rf_model.feature_importances_
ax = axes[2]
sorted_idx = np.argsort(rf_importance)
ax.barh(range(len(feature_names)), rf_importance[sorted_idx], color=colors['
ax.set_yticks(range(len(feature_names)))
ax.set_yticklabels([feature_names[i] for i in sorted_idx])
ax.set_xlabel('Importance')
ax.set_title('Random Forest')

plt.suptitle('Feature Importance Comparison', fontsize=14)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'feature_importance.png'), dpi=150, bb
plt.show()
```



11f. Loss Curves

```
fig, ax = plt.subplots(1, 1, figsize=(8, 6))

# XGBoost eval results
xgb_evals = xgb_model.evals_result()
if xgb_evals and 'validation_0' in xgb_evals:
    val_loss = xgb_evals['validation_0']['logloss']
    ax.plot(range(len(val_loss)), val_loss, color=colors['XGBoost'], label='

ax.set_xlabel('Iteration')
ax.set_ylabel('Log Loss')
ax.set_title('Training/Validation Loss Curves')
ax.legend()
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'loss_curves.png'), dpi=150, bbox_inch
plt.show()
```



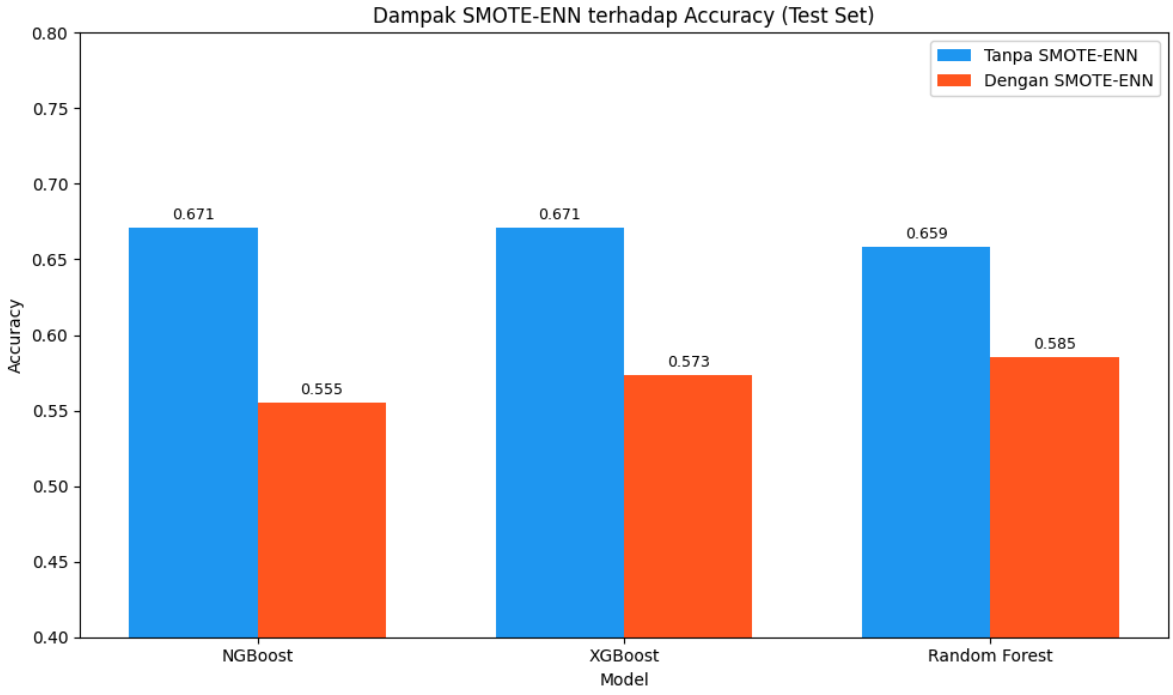
11g. SMOTE-ENN Comparison

```
# Train all models with SMOTE-ENN for full comparison
print("Training models WITH SMOTE-ENN for comparison...")
```

```
ngb_smote = NGBClassifier(Dist=Bernoulli, n_estimators=300, learning_rate=0.
                          minibatch_frac=0.8, col_sample=0.8, random_state=R
ngb_smote.fit(X_train_resampled, y_train_resampled)
ngb_smote_acc = accuracy_score(y_test, ngb_smote.predict(X_test_scaled))

xgb_smote = XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=4,
                          subsample=0.8, colsample_bytree=0.8, random_state
                          eval_metric='logloss', use_label_encoder=False, v
xgb_smote.fit(X_train_resampled, y_train_resampled)
xgb_smote_acc = accuracy_score(y_test, xgb_smote.predict(X_test_scaled))

rf_smote = RandomForestClassifier(n_estimators=300, random_state=RANDOM_STAT
rf_smote.fit(X_train_resampled, y_train_resampled)
rf_smote_acc = accuracy_score(y_test, rf_smote.predict(X_test_scaled))

print(f"  NGBoost:       {results['NGBoost']['accuracy']:.4f} (tanpa) vs {ng
print(f"  XGBoost:       {results['XGBoost']['accuracy']:.4f} (tanpa) vs {xg
print(f"  Random Forest: {results['Random Forest']['accuracy']:.4f} (tanpa)
```

```
Training models WITH SMOTE-ENN for comparison...
  NGBoost:       0.6707 (tanpa) vs 0.5549 (dengan)
  XGBoost:       0.6707 (tanpa) vs 0.5732 (dengan)
  Random Forest: 0.6585 (tanpa) vs 0.5854 (dengan)
```

```
fig, ax = plt.subplots(1, 1, figsize=(10, 6))

x_pos = np.arange(3)
width = 0.35
no_smote_accs = [results['NGBoost']['accuracy'], results['XGBoost']['accurac
smote_accs = [ngb_smote_acc, xgb_smote_acc, rf_smote_acc]

bars1 = ax.bar(x_pos - width/2, no_smote_accs, width, label='Tanpa SMOTE-ENN
bars2 = ax.bar(x_pos + width/2, smote_accs, width, label='Dengan SMOTE-ENN',

ax.set_xlabel('Model')
ax.set_ylabel('Accuracy')
ax.set_title('Dampak SMOTE-ENN terhadap Accuracy (Test Set)')
ax.set_xticks(x_pos)
ax.set_xticklabels(['NGBoost', 'XGBoost', 'Random Forest'])
ax.legend()
ax.set_ylim([0.4, 0.8])

for bar in bars1:
    height = bar.get_height()
    ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width() / 2, heig
                xytext=(0, 3), textcoords="offset points", ha='center', va='
for bar in bars2:
    height = bar.get_height()
    ax.annotate(f'{height:.3f}', xy=(bar.get_x() + bar.get_width() / 2, heig
                xytext=(0, 3), textcoords="offset points", ha='center', va='

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, 'smote_enn_comparison.png'), dpi=150,
plt.show()
```



12. Ringkasan Hasil untuk Paper

Berikut adalah ringkasan lengkap semua hasil eksperimen yang dapat langsung digunakan untuk memperbarui paper.

```
print("=" * 70)
print("RINGKASAN HASIL EKSPERIMEN")
print("=" * 70)

print(f"""
DATASET:
  - Total samples: {len(df)}
  - Features: {len(df.columns) - 1}
  - Class distribution: {class_dist[0]} ({class_dist[0]/len(df)*100:.2f}%)

MISSING VALUES:
  - ph: {(df['ph'].isnull().sum()/len(df)*100):.2f}%
  - Sulfate: {(df['Sulfate'].isnull().sum()/len(df)*100):.2f}%
```

```
  - Trihalomethanes: {(df['Trihalomethanes'].isnull().sum()/len(df)*100):.2

DATA SPLIT:
  - Training: {X_train.shape[0]} samples
  - Validation: {X_val.shape[0]} samples
  - Test: {X_test.shape[0]} samples

SMOTE-ENN EVALUATION:
  - Before: {X_train_scaled.shape[0]} samples
  - After: {X_train_resampled.shape[0]} samples (REDUCED by {X_train_scaled
  - KEPUTUSAN: TIDAK diterapkan (degrades performance)

FINAL RESULTS (Test Set, N={len(y_test)}):
""")

print(f"{'='*60}")
print(f"{'Metric':<15} {'NGBoost':<15} {'XGBoost':<15} {'Random Forest':<15
print(f"{'='*60}")
for metric in ['accuracy', 'precision', 'recall', 'f1', 'nll', 'ece', 'auc'
    label = metric.upper() if metric in ['nll', 'ece', 'auc'] else metric.c
    print(f"{label:<15} "
          f"{results['NGB'][metric]:<15.4f} "
```